

MIDTERM 1 SOLUTIONS

Q1. (Total of 33 points)

a) (4p) Show using DeMorgan's Theorem that



De Morgan's Theorem states that

$$(A \cdot B)' = A' + B'.$$

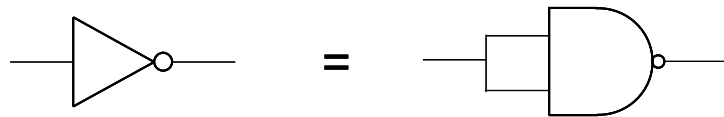
The left-hand side of this equation is the standard NAND definition, and the right-hand side is the alternative representation depicted in the above symbol.

b) (6p) Prove using the canonical SOP representation of a combinational logic function that NAND gates are sufficient to implement any combinational logic circuit.

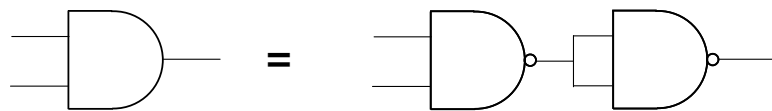
Method I:

Any combinational logic function can be represented in Canonical SOP form. It is trivially straightforward to implement the Canonical SOP form using inverters (to implement complemented literals), AND gates (to implement the products), and OR gates (to implement sums). It is sufficient to show that we can implement each of inverter, AND, and OR gate using NAND gates.

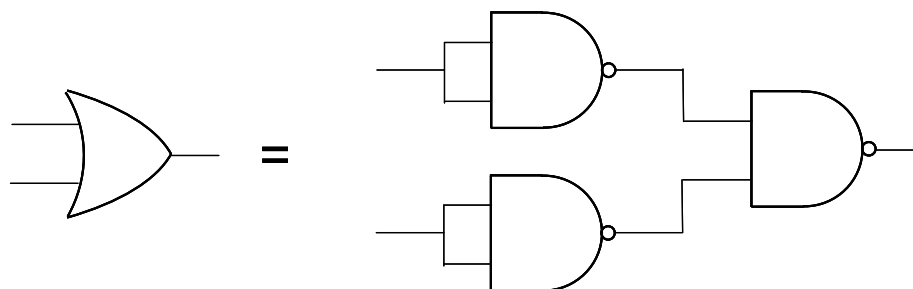
Inverter:



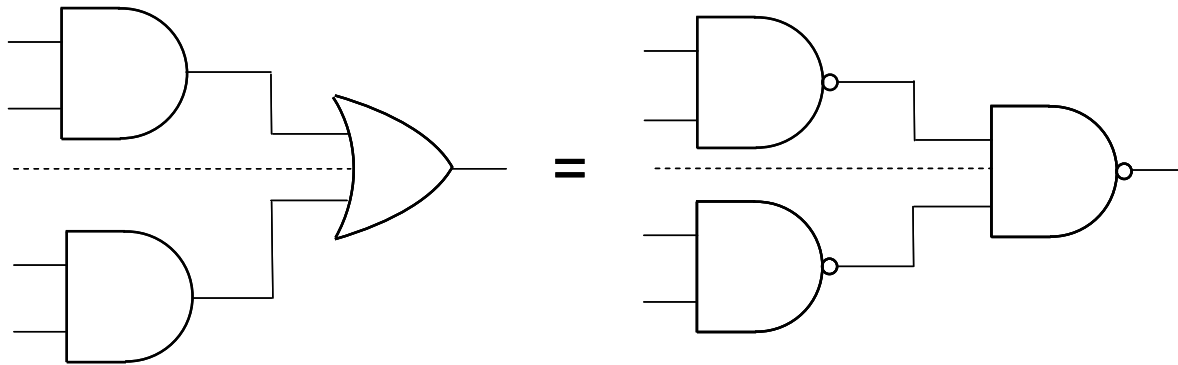
AND:



OR:



Method II: Show inverter implementation as in Method II. Instead of showing AND and OR implementation independently, just show the implementation of AND-OR combination used in Canonical SOP form.



c) (4p) Find the complement of the function $F = A + BC'(D + E + 1)$. Do not simplify the expression.

We apply De Morgan's Rule repeatedly.

$$F' = (A + BC'(D+E+1))' = A' \cdot (BC'(D+E+1))' = A' \cdot (B' + C + (D+E+1)') \\ = A' \cdot (B' + C + (D' \cdot E' \cdot 0))$$

d) (4p) Find the dual of the function $F = A + BC'(D + E + 1)$. Do not simplify the expression.

$$F^D = (A + (BC'(D+E+1)))^D = (A \cdot (B + C' + (D \cdot E \cdot 0)))$$

e) (4p) Expand the function $F = A B + CD$ with respect to A using Shannon's theorem.

$$F(A,B,C) = A' \cdot F(0,B,C) + A \cdot F(1,B,C) = A' \cdot (C \cdot D) + A \cdot (B + C \cdot D)$$

f) (5p) Add out the function $F = A + BC + D$ to obtain a POS form.

$$F = A + BC + D = (A + B)(A + C) + D = (A + B + D)(A + C + D)$$

g) (6p) The Boolean variables A , B , and C have values that we don't know, but we know that $AB = AC$. If $C = 0$, what can we say about B .

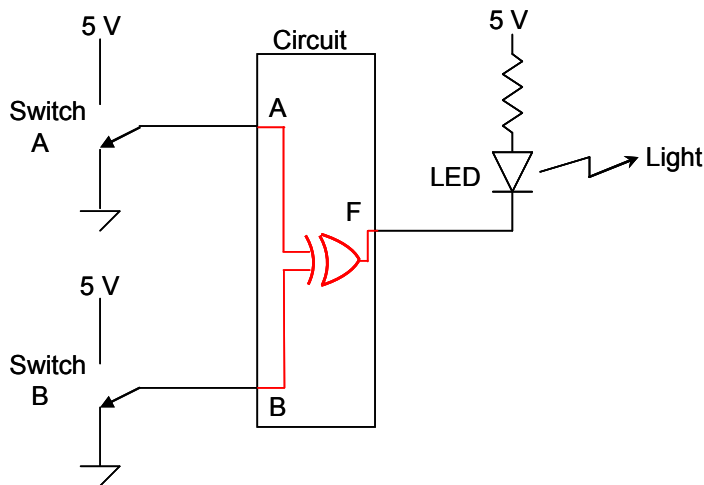
If $A=1$, then $B=C$, which means $B=0$.

If $A=0$, then both sides of $AB = AC$ will be 0, regardless of the values of B and C . In this case, we cannot say anything about the value of B . (It can be 0 or 1)

In short, without knowing the value of A , we cannot say anything about the value of B .

Q2. (15 points)

A “two-way switch” system is used at homes to turn a light ON or OFF from two switches located in two different places.



If switch A is pulled UP then the signal A becomes 1 and if it is pulled DOWN then the signal A becomes 0. Similarly for switch B.

Now suppose switch A is located at the bottom of a stairway and switch B is located at the top. The LED (Light Emitting Diode) is used to illuminate the stairway. Assume, as a scenario, that the LED is OFF at the beginning. A person who wants to climb up the stairs toggles switch A to turn the light ON. He then climbs up the stairs and at the top he toggles switch B to turn the light back OFF. This scenario is given as an example to explain how the system is used.

Design and draw the inside of the “Circuit” using as few gates as possible.

Notes: 1) For the LED to be ON, F must be 0. To turn it OFF, F must be 1.

2) “Toggle” means “change the position of the switch”.

3) The initial positions of the switches are unknown although they are drawn in the above figure as at DOWN position.

Assume that both switches are DOWN, and LED is OFF at the beginning. This gives “ $F = 1$ when $A=0$ and $B=0$ ”. Next, we should switch on the light by toggling either one of A or B. This gives “ $F=0$ when $A=1$ and $B=0$ ”, and “ $F=0$ when $A=0$ and $B=1$ ”. Finally, we should switch off when the light is ON by toggling either one of A or B. This gives “ $F=1$ when $A=1$ and $B=1$ ” and “ $F=1$ when $A=0$ and $B=0$ ”. The latter was already covered by the initial case. Combining the givens into a truth table, we have:

A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

This is the truth table of an XNOR (equivalence) gate.

With the same reasoning and different initial conditions (e.g. “both switches DOWN and LED is ON”, or “one of the switches UP, one of the switches DOWN, and LED is ON”), we will end up with the truth table of an XOR gate.

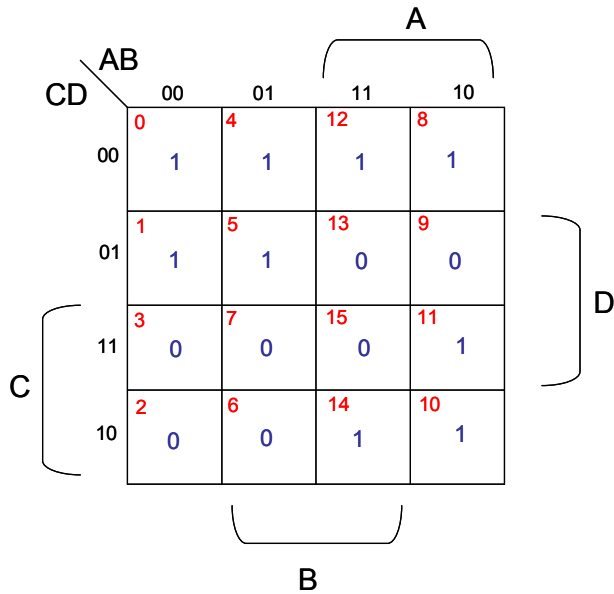
Note that in practice initial conditions do not matter: We build the circuit by using one XOR gate or one XNOR gate, and if the LED is ON, we switch it off by toggling one of the switches.

Q3. (Total of 24 points)

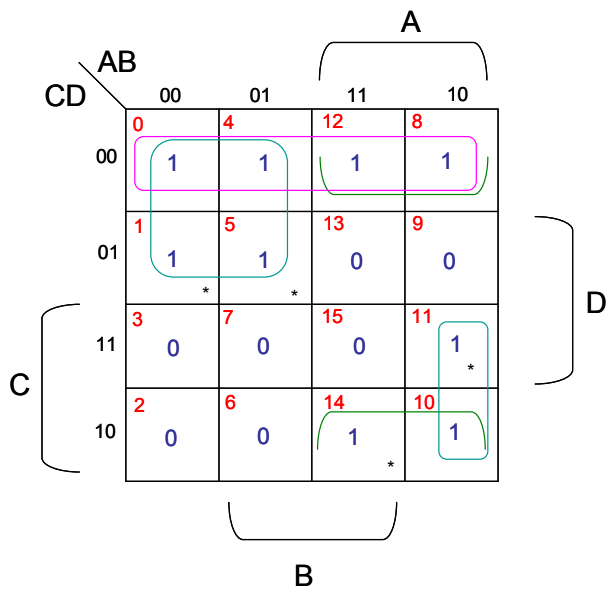
Use the Karnaugh Map method to answer the following questions for the function

$$F = A'B'C'D' + A'BC'D' + ABC'D' + AB'C'D' + A'B'C'D + A'BC'D + AB'CD + ABCD' + AB'CD'$$

a) (5p) Draw the Karnaugh map for F



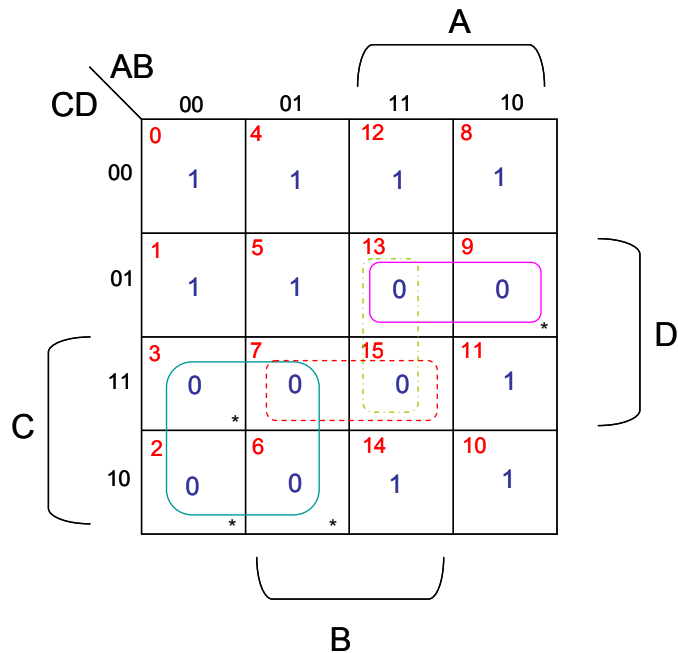
b) (7p) Find a minimal SOP expressions for F.



In this case, essential prime implicants cover all ones.

$$F = A' C' + A D' + A B' C$$

c) (7p) Find a minimal POS expressions for F.



In this case, after essential prime implicants are chosen, only one 0-cell is left uncovered (cell 15). This cell can be covered by the maxterm $(A' + B' + D')$ which is depicted with dash-dot in the graph, or by the maxterm $(B' + C' + D')$ which is depicted with dash in the graph. Both choices are equally good.

$$F = (A + C') (A' + C + D') (A' + B' + D')$$

or

$$F = (A + C') (A' + C + D') (B' + C' + D')$$

d) (5p) Which of the above found expressions would you use for implementation. Why?

SOP form has 3 terms and 7 literals. POS form has 3 terms and 8 literals. We would use SOP form, since it has no more terms and fewer literals, thus would result in fewer or smaller gates.

Note: If your Karnaugh map is not correct, you may not get points for the rest of your solution.

Q4. (20 points)

An odd-parity module (entity) “odd_parity_8bit” for 8-bit binary numbers is already written using VHDL and saved.

Write VHDL code for the entity named “odd_parity_16bit” which describes an odd_parity circuit for 16-bit binary numbers. Use “odd_parity_8bit” module in your code.

The entity declaration of “odd_parity_8bit” is as follows:

```
entity odd_parity_8bit is
    port (A:in std_logic_vector (7 downto 0);
          F: out std_logic);
end odd_parity_8bit;
```

Notes: 1) What “odd_parity_8bit” does: If the number of 1 bits in A is odd then F = 1, else F = 0.

2) If X is declared as std_logic_vector (7 downto 0) then X(5 downto 1) is a 5-bit number picking the appropriate subrange of X.

3) It is not important for this exam’s purposes to include all necessary libraries.

4) Do not write “odd_parity_16bit” from scratch but make use of the previously written and saved “odd_parity_8bit”.

```
entity odd_parity_16bit is
    port (A:in std_logic_vector (15 downto 0);
          F: out std_logic);
end odd_parity_16bit;

architecture structural of odd_parity_16bit is

    component odd_parity_8bit
        port (A:in std_logic_vector (15 downto 0);
              F: out std_logic);
    end component;

    signal F8H, F8L: std_logic;

begin
    label1: odd_parity_8bit port map(A(15 downto 8), F8H);
    label2: odd_parity_8bit port map(A(7 downto 0), F8L);

    F <= F8H xor F8L;
End structural;
```

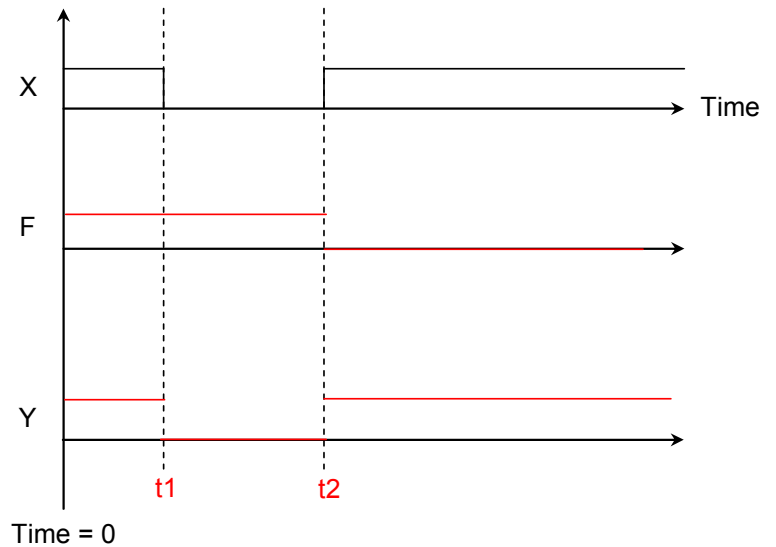
Q5. (15 points)

Draw the timing diagram for the circuit described by the following VHDL code for the input signal waveform given below.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity simple is
    port(X:in std_logic;
          F:out std_logic);
end simple;

architecture Behavioral of simple is
    signal Y:std_logic := '1';
begin
    process(X)
    begin
        Y<=X;
        F<=Y;
    end process;
end Behavioral;
```



Note that Y is not in the sensitivity list of the process. The process will be executed once at t=0, and then whenever the value of X changes, namely at t=t1 and t=t2. Since X is not in the left hand side within the process, the process will be executed at most once for each t.

Remember that the new values of the assigned signals become effective after the process call terminates. For this reason, the statement $F \leq Y$ will store the **old value** of Y to F each time it executes. At t=0, the old value of Y is 1, due to the initialization statement. At t=t1, the old value of Y is 1. At t=t2, the old value of Y is 0.

Q6. (Total of 18 points)

Implement the function $F = AB + B'C$

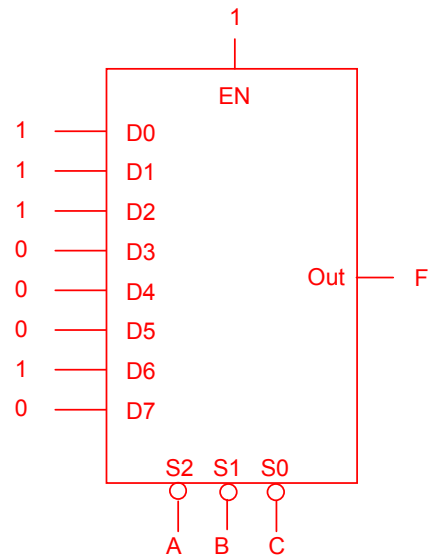
A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- a) (6p) using one 8-to-1 multiplexer with active_low selects (control inputs) only, (You are not allowed to use any extra gate. Draw your solution and label all inputs and outputs.)

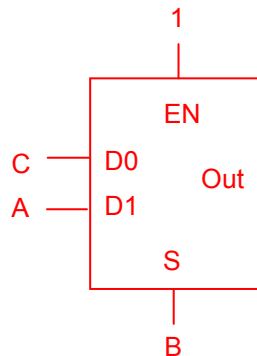
We need to reorder the inputs in order to compensate for the active-low select inputs. The reordering is:

New_index = 7 - TT row index

This is the same thing as applying the truth table "upside-down".



- b) (6p) using one generic 2-to-1 multiplexer only, and (You are not allowed to use any extra gate. Draw your solution and label all inputs and outputs.)



c) (6p) using one 74x138 3-to-8 decoder and a single multi-input gate.

